

Pintos **ASSERT**

깨져서는 안 되는 조건을 코드로 남기기

왜 추천하나?

```
Kernel PANIC at ../../threads/synch.c:237 in lock_acquire():  
assertion `!lock_held_by_current_thread (lock)' failed.  
Call stack: 0x800426eff 0x8004202fb ...
```

```
0x0000000800426eff: debug_panic (lib/debug.c:86)  
0x00000008004202fb: file_seek (filesys/file.c:405)  
0x000000080042dc22: seek (userprog/syscall.c:744)  
0x000000080042cf67: syscall_handler (userprog/syscall.c:444)  
...
```

깨진 조건과 호출 경로를 바로 보여준다.

어디에 쓰나?

pintos/lib/kernel/list.c - list_sort()

```
void
list_sort (struct list *list, list_less_func *less, void *aux) {
    ASSERT (list != NULL);
    ASSERT (less != NULL);

    ...

    ASSERT (is_sorted (list_begin (list),
                        list_end (list), less, aux));
}
```

선조건과 후조건을 실제 코드에서 검사한다.

내 코드 예시

```
void  
threads_wakeup (int64_t ticks) {  
    ASSERT (intr_context ());  
    ASSERT (intr_get_level () == INTR_OFF);  
  
    ...  
}
```

timer interrupt 안에서만 호출된다는 전제를 검증한다.

템플에서 좋은 점 1

```
void  
threads_wakeup (int64_t ticks) {  
    // interrupt handler에서만 호출되어야 한다.  
    // interrupt는 꺼진 상태여야 한다.  
  
    ...  
}
```

주석은 의도를 설명하지만, 실행 중에 검증하지는 않는다.

템플에서 좋은 점 2

```
void  
threads_wakeup (int64_t ticks) {  
    ASSERT (intr_context ());  
    ASSERT (intr_get_level () == INTR_OFF);  
  
    ...  
}
```

ASSERT 는 의도를 코드로 보여주고, 잘못 호출하면 바로 실패한다.

주의점

```
int index = 1;  
  
printf ("before: %d\n", index);  
ASSERT (index++ > 0);  
printf ("after: %d\n", index);
```

```
ASSERT 켜짐: before: 1 -> after: 2  
ASSERT 꺼짐: before: 1 -> after: 1
```

상태 변경이 `ASSERT` 안에 있으면 빌드 설정에 따라 결과가 바뀐다.

결론

ASSERT 를 많이 쓰자.

프로젝트 회고: AI 사용 기준

먼저 목표를 정하자

- 내 목표: 가능한 한 전체 코드를 직접 구현하기
- AI 활용: 요구사항 분석, 막혔을 때 확인할 방향 잡기
- 직접 수행: 코드 작성, 리팩토링, 최종 판단